Leakage, Poisoning and Evasion: Auditing a Machine-Learning Intrusion Detector on UNSW-NB15

Guerfi Ahmed Yacine
École Supérieure en Informatique de Sidi Bel Abbès (ESI-SBA), Algeria
ay.guerfi@esi-sba.dz github.com/N4khlA/AI-_threat_hunting

Technical report, November 2026

Abstract

Machine-learning intrusion detectors are usually judged by one number on a benchmark split, and treated as trusted components once deployed. This report audits both assumptions on a Random Forest flow classifier trained on UNSW-NB15. First, the evaluation: about 40% of the dataset's rows are exact duplicates, and a random re-split puts copies of the same flow in training and test. Moving to the official split, with test flows restricted to those never seen in training, lowers macro-F1 from 0.948 to 0.818 and raises the false-positive rate at threshold 0.5 from 5.3% to 26.5%; the same leak inflated a BiLSTM's AUC to 0.998. Second, the detector as a target. An unauthenticated label field in the scoring API allowed training-data poisoning: 2% poisoned rows cut Backdoor recall from 1.00 to 0.67 while overall accuracy rose, so an aggregate-metric retrain guard accepted the model. On evasion, an attacker who only chooses their initial IP TTL evades the plainly trained model on 100% of detected attacks, because TTL is a testbed artifact that the model uses as a shortcut. Dropping the TTL features and adding adversarial training reduces evasion to at most 3.3% across all budgets tested, at a cost of 0.005 macro-F1. Every number is regenerated by a script in the repository.

1 Introduction

Benchmark results for ML-based network intrusion detection are often near-perfect, yet these systems are rarely trusted in production. Sommer and Paxson [2] and Arp et al. [3] attribute much of this gap to evaluation pitfalls such as data snooping, spurious correlations and unrealistic base rates; Kapoor and Narayanan [4] show the same leakage problems across ML-based science. A second gap is that a deployed detector is itself software with an attack surface: its training data, its retraining loop and its inputs can all be manipulated [5].

This project started as a conventional detector (v1) that reported macro-F1 0.948 for a Random Forest and AUC 0.998 for a BiLSTM on UNSW-NB15 [1]. This report documents a systematic audit of that system and the fixes that followed. Its contributions are:

1. An evaluation audit across five split protocols that isolates how much of the headline score came from duplicates and split choice (§3), and what the remaining performance means at realistic base rates (§4).
2. A label-poisoning proof of concept against the system's own retraining loop, and a per-family guard that blocks it (§5).
3. A black-box evasion benchmark with physically plausible, attacker-side perturbations, which exposes TTL as a shortcut feature (§6).
4. A hardened model (no TTL features, adversarial training) that is now the one served, with its accuracy cost measured (§7).

2 System and data

Dataset. UNSW-NB15 [1] contains network flows from a 2015 testbed, described by 49 Argus/Bro features (45 numeric, 4 categorical), with nine attack families. We use the cleaned CSV release (257,673 records, 63.9% attacks), which provides an official training file and an official testing file captured in different periods, but no IP addresses or timestamps.

Model and pipeline. The main detector is a Random Forest (100 trees, unlimited depth, `min_samples_split`=5) trained on imputed, encoded and scaled features with SMOTE oversampling on the training set only. A separate Random Forest classifies the attack family for alerting, and a bidirectional LSTM scores windows of 10 consecutive flows. The model is served by a FastAPI service that emits alerts with a severity, a MITRE ATT&CK mapping and the exact per-feature decision-path contributions of the forest.

Operating point. Unless stated otherwise, thresholds are chosen on a stratified 15% validation split of the official training file so that validation FPR $\leq 1\%$, then applied unchanged to the test set.

3 Evaluation leakage

The v1 pipeline merged both files and re-split them at random 70/15/15. Because 40% of rows are exact duplicates, many test flows had identical twins in training. We evaluated the same model and hyperparameters under five protocols (Table 1); 95% confidence intervals are percentile bootstraps over test rows.

Table 1: One Random Forest, five evaluation protocols. *Novel* restricts the test set to flows with no exact copy in training (74,072 duplicate training rows, 28,380 duplicate test rows and 1,304 test rows seen in training removed). The last row is the served model.

Protocol	Test rows	Macro-F1 [95% CI]	ROC-AUC	FPR@0.5
Random re-split (v1)	38,651	0.948 [0.945, 0.950]	0.992	5.3%
Random re-split, deduplicated	23,115	0.917 [0.914, 0.922]	0.981	7.6%
Official train → test	82,332	0.862 [0.860, 0.865]	0.981	27.5%
Official, novel test	52,648	**0.818** [0.815, 0.822]	0.963	26.5%
Official, novel, no TTL features	52,648	0.816 [0.813, 0.819]	0.961	26.8%
Official, novel, hardened (served)	52,648	0.813 [0.810, 0.817]	0.960	27.5%

Three observations follow.

- **The split, not the model, explains about 13 F1 points.** Deduplication alone costs 3 points; testing on the other capture period costs most of the rest, and the false-positive rate at 0.5 rises five-fold because benign traffic differs between captures.
- **Feature importance is not feature necessity.** The two most important features are `sttl` and `ct_state_ttl`, both derived from the IP time-to-live, a known artifact of the UNSW-NB15 testbed. Removing all three TTL features costs only 0.002 F1, since other features carry the same separation. Section 6 shows that keeping them is nonetheless costly.
- **Thresholds do not transfer.** A threshold chosen for 1% FPR on validation yields 4–5% FPR on test.

The same leak affected the sequence model: with a stride of 5 and windows of 10, the v1 random window split placed 54,041 flows in both training and test windows. Building windows per file removes the overlap and lowers the LSTM's AUC from 0.998 to 0.989 and its F1 from 0.985 to 0.894. On the 10,544 flows both models can score, the LSTM and the Random Forest

are statistically tied (accuracy 0.819 vs. 0.815, McNemar $p = 0.27$). The v1 conclusion that the LSTM was stronger was an artifact of the leak.

4 What the remaining performance means in practice

At its operating threshold (0.880) the served model detects 76.5% of attacks at 4.3% FPR. Following Axelsson [7], we translate this into alert volumes for a network producing one million flows per day (Table 2).

Table 2: Expected alerts per day for 10^6 flows at the served operating point.

Attack prevalence	Alert precision	True alerts/day	False alerts/day
1%	15.2%	7,654	42,551
0.1%	1.8%	765	42,938

A model with 0.96 ROC-AUC still produces about 43,000 false alerts a day. The system is therefore designed for human triage rather than automatic blocking: alerts are ranked by severity and carry the features that drove the verdict. Per-family recall at this threshold ranges from 0.96 (Generic) to 0.23 (Fuzzers); Fuzzers traffic is designed to resemble benign flows and remains the main weakness.

5 Attack 1: label poisoning

Vulnerability. The v1 `/predict` endpoint accepted optional `Label` and `attack_cat` fields from any caller, stored them as ground truth, and the retraining script trained on them (MITRE ATLAS [8] AML.T0020). A candidate model replaced the incumbent if its accuracy or F1 improved.

Experiment. The incumbent is a Random Forest trained on 60% of the training set (accuracy 0.816, Backdoor recall 1.00). Each candidate is trained on the full training set, which also contains genuine new data, plus n attacker-submitted copies of Backdoor flows labelled benign.

Table 3: Targeted poisoning of the Backdoor family. Accuracy stays flat or improves, so an aggregate-only guard accepts every poisoned model.

Poison budget	Rows	Accuracy	Backdoor recall	v1 guard	v2 guard
0%	0	0.8208	1.000	accept	accept
0.25%	221	0.8204	1.000	accept	accept
0.5%	441	0.8203	0.957	accept	refuse
1%	882	0.8196	0.829	accept	refuse
2%	1,764	0.8210	0.671	accept	refuse
5%	4,411	0.8181	0.614	accept	refuse

Result. Table 3: with 2% poisoned rows, one in three Backdoor flows goes undetected, while accuracy is higher than with no poison at all. Because the candidate also benefits from genuine new data, the v1 guard accepted every poisoned model.

Fixes. (i) The scoring schema rejects unknown fields, so a `Label` on `/predict` returns HTTP 422. (ii) Labels are accepted only through a separate `/feedback` endpoint that requires an analyst API key, references a logged prediction, and records who labelled it. (iii) Retraining uses only analyst labels; pseudo-labelling is opt-in. (iv) The v2 guard refuses any candidate whose recall on any

attack family drops by more than 0.02 relative to the incumbent, which catches the attack from the 0.5% budget onwards.

6 Attack 2: evasion

Threat model. A black-box attacker (ATLAS AML.T0015 / AML.T0043) observes only whether a flow is flagged and controls only their own side of the connection. Every perturbation adds cost without removing the attack: padding payloads (`sbytes`), adding packets (`Spkts`, with header bytes), slowing the flow (`dur` and inter-packet gaps), and choosing the initial TTL, set to a value typical of benign traffic. Derived features (`Sload`, `Dload`, `smeansz`, `Sintpkt`) are recomputed so that records stay self-consistent; destination-side fields are untouched. Each continuous knob is drawn from $[0, \varepsilon]$ as a relative increase, with 40 random-search queries per flow. We attack 1,500 correctly detected test attacks per model; success means the score falls below the model's operating threshold.

Models. *Plain*: the v1 recipe on all features. *No-TTL*: the same without `sttl`, `dttl` and `ct_state_ttl`. *Hardened*: no TTL plus adversarial training (§7). The three have nearly identical clean performance (TPR 0.787 / 0.779 / 0.778 at FPR 0.050 / 0.048 / 0.050).

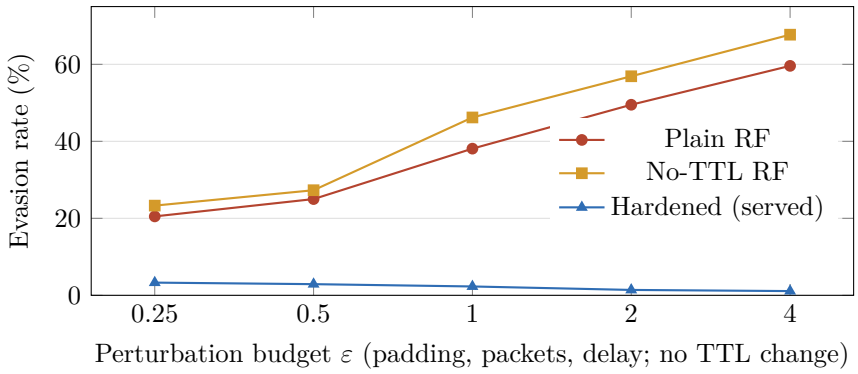


Figure 1: Share of detected attacks pushed below the operating threshold. At $\varepsilon = 0$ every model is at 0%, which checks that the perturbation code is an exact identity at zero budget.

The TTL shortcut. In UNSW-NB15, an observed `sttl` of 31 occurs only in benign rows. An attacker who sets their initial TTL so that the sensor observes this value evades the plain model on **100%** of detected attacks, at every budget including $\varepsilon = 0$. A single attacker-controlled header field defeats the model. Section 3 showed that the TTL features are barely needed for accuracy; this experiment shows they must be removed for robustness. Importance analysis alone would not have revealed this.

Cheap perturbations. Without TTL, padding and slowing still evade 23–68% of detections (Figure 1). The no-TTL model is slightly *easier* to evade with them than the plain one, presumably because without the shortcut it leans more on the volume and timing features that these perturbations change. Fuzzers (89% evaded at $\varepsilon = 1$) and Analysis (74%) are the easiest families to push over.

7 Hardening

The served model combines two measures: it never sees the TTL features, and its training set is augmented with perturbed copies of every training attack (knobs drawn from $U(0, 2)$, TTL drawn

from benign-typical values), still labelled as attacks [6]. The same module builds the training set for initial training, retraining and evaluation, so the recipe cannot drift between them.

Table 4: Evasion rate with TTL rewriting allowed, and clean cost of hardening.

Model	$\varepsilon = 0$	$\varepsilon = 0.25$	$\varepsilon = 1$	$\varepsilon = 4$	Macro-F1 (novel test)
Plain RF	100%	100%	100%	100%	0.818
No-TTL RF	0%	23.1%	46.2%	66.7%	0.816
Hardened	0%	**3.2%**	**1.9%**	**1.3%**	0.813

Hardening reduces evasion to at most 3.3% at every budget tested, with or without TTL rewriting, for a cost of 0.005 macro-F1 (Table 4). The worst remaining family is Fuzzers, at 10.5% evasion for $\varepsilon = 1$. A regression test checks that changing `sttl` cannot change the served model's score. This robustness holds only for the perturbation family the model was trained on; Section 8 lists what is not covered.

Service hardening. Beyond the model, the API was hardened against the OWASP API Security Top 10: role-based API keys stored as SHA-256 digests and compared in constant time, failing closed when none are configured; a strict input schema that rejects unknown fields, NaN and out-of-range values; per-key rate limiting and body-size caps; and SHA-256 pinning of every model artifact and of the alert threshold, verified at startup on the exact bytes that are then deserialised. The repository contains 108 automated tests, with static analysis (`bandit`) and dependency auditing (`pip-audit`) in continuous integration.

8 Limitations

- **One synthetic dataset.** UNSW-NB15 is a 2015 testbed. No cross-dataset evaluation was run: the features of datasets such as CIC-IDS2017 barely overlap with UNSW-NB15's, so a transfer score would mostly measure feature mapping. The shift between the two official capture periods is the closest available proxy.
- **Weak sequence signal.** The cleaned release has no IP addresses or timestamps, so LSTM windows group flows by protocol and service in file order. Per-host sequences need the original files.
- **Limited adversary.** The evasion attacker uses random search over four knobs. An adaptive attacker who changes protocol, splits an attack across many flows, or optimises against the hardened model directly is out of scope.
- **No end-to-end capture.** The models need Argus features; the demo replays held-out flows rather than processing raw packet captures.
- **Operational gaps.** Thresholds must be recalibrated on the defended network, and an attacker able to rewrite both an artifact and its manifest defeats pinning unless the manifest is signed.

9 Conclusion

Most of this detector's original headline score came from the evaluation, not the model: duplicates and a random split accounted for about 13 F1 points, and a sequence model's apparent advantage disappeared once training and test windows stopped overlapping. Treating the detector as a target exposed two further failures that standard metrics hide: a poisoning path through the retraining loop that left accuracy unchanged, and a testbed artifact that let one header field evade every detection. Both were fixed at a measured cost of 0.005 macro-F1. The broader lesson is that

leakage audits, per-family metrics and adversarial testing belong in the evaluation of security ML systems, not after it.

Reproducibility. `scripts/reproduce.sh` regenerates every table and figure in about 15 minutes on 8 CPU cores with fixed seeds. Each report in `reports/` names the script that produced it (`rigorous_eval.py`, `poisoning.py`, `evasion.py`, `evaluate.py`).

References

[1] N. Moustafa and J. Slay. UNSW-NB15: A comprehensive data set for network intrusion detection systems. In *Military Communications and Information Systems Conference (MilCIS)*, 2015.

[2] R. Sommer and V. Paxson. Outside the closed world: On using machine learning for network intrusion detection. In *IEEE Symposium on Security and Privacy*, 2010.

[3] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck. Dos and don'ts of machine learning in computer security. In *USENIX Security Symposium*, 2022.

[4] S. Kapoor and A. Narayanan. Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 4(9), 2023.

[5] B. Biggio and F. Roli. Wild patterns: Ten years after the rise of adversarial machine learning. *Pattern Recognition*, 84, 2018.

[6] I. Goodfellow, J. Shlens, and C. Szegedy. Explaining and harnessing adversarial examples. In *International Conference on Learning Representations (ICLR)*, 2015.

[7] S. Axelsson. The base-rate fallacy and the difficulty of intrusion detection. *ACM Transactions on Information and System Security*, 3(3), 2000.

[8] MITRE. ATLAS: Adversarial Threat Landscape for Artificial-Intelligence Systems. `https://atlas.mitre.org`.